

Mid Review

Multiple Choice

1. What is the output of the following program?

Process and system calls

```
#include<stdio.h> #include<stdlib.h>
#include <unistd.h>
int main() {
```

```
    pid_t child;
```

```
    child Parent
```

```
    int status;
```

```
    Child fork();
```

```
    switch (child) {
```

```
    case -1:
```

```
        error perror("fork");
        exit(1);
```

```
    case 0:
```

```
        Child → exit (2);
        break;
```

```
    default:
```

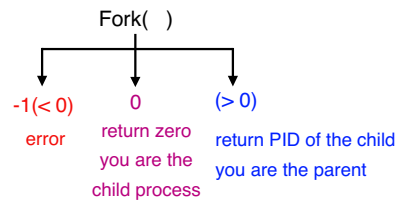
```
        Parent → while (waitpid (child, &status, WNOHANG) == 0)
```

```
            sleep(1);
```

```
            printf("%d\n", WEXITSTATUS (status));
```

```
        }
```

```
    return 0;
```



While Child is alive

If the child is died it
will print exit status

exit code for the child = 2

a) 0

b) 1

c) 2

d) None of the above

2. What is the output of the following program?

Process and system calls

```
#include<stdio.h>
#include <unistd.h>
```

```
int main() {
```

```
    int x=10, y = 80;
```

```
    y= y + x; → y = 90
```

```
    pid t c fork();
```

```
    if (c == 0) {
```

```
        Child x += y; → x = x+y = 10 + 90 = 100
```

```
        printf("%d%d", x, x - y);
```

100 100 - 90 = 10

10010

```
    } else {
```

```
        Parent wait (NULL);
```

```
        printf("%d%d", x, y);
```

10 90

1090

```
    return 0;
```

```
}
```

a) 100109090

b) 100101090

c) 109010010

d) 1001010090

3. How many times "CMPS405" will be printed in this program?

Process and system calls

```
#include<stdio.h>
#include<stdlib.h>
#include <unistd.h>
int main() {
    if (fork()==0)
        if (execl("/bin/ls", "ls", NULL) != -1) {
            perror("execl");
            exit(1);
        }
    printf("CMPS405\n");
    return 0;
}
```

Child

a) 0

b) 1

c) 2

d) None of the above

4. This is C program that forks as many sibling children as needed to print a message entered by the user. And each process must print exactly one character of the message. The message characters must appear in the correct order. The following code should avoid any premature termination or zombie processes.

Process and system calls

```
void main( ) {
    Pid_t pid; char msg[n]; int n;
    printf("Enter a message: ");
    fgets(msg);

    for (int i = 0; i < strlen(msg); i++) {
        If (! fork()) {
            printf("%c" , msg[i]);
            exit(0);
        }
        else
            wait(NULL);
    }
    return 0;
}
```

- a) The code is fully correct and meets all requirements.
- b) The code prints characters in order but contains syntax and declaration errors.
- c) The code creates zombies because the parent does not wait.
- d) The code prints the message in reverse order.

5. In the Additional-Reference-Bits Algorithm used by LRU, which of the following shift register values represents a page that would be the best choice for replacement? Virtual Memory

a) 00110011

b) 01010101

c) 10101010 ✗

d) 11001100 ✗

Additional References start with zero.

Enhanced second chance.

6. Which of the following is an advantage of the Layered Structure of OS Design? Introduction

a) Defining appropriate layers data and operations is an easy task —————→ Disadvantage

b) Direct user access to low-level facilities is allowed

c) Very efficient in communicating requests between the layers

d) **Simplification of debugging and system verification**

7. Which type of Operating System architecture allows all processors to communicate with each other by a shared memory? Introduction

a) **Symmetric multiprocessing**

b) Asymmetric multiprocessing

c) Real Time

d) Distributed

8. Which of the following describes a Zombie process? Process and system calls

a) A process that its parent exited while it is still running

b) **A process that makes exit when its parent is not executing wait**

c) A process that is adopted by the system init process

d) A process that replaces its memory space with a new program

9. Which address-binding scheme is true for logical and physical addresses ? Main Memory

a) **Same in compile-time and load-time, but differ in execution-time.**

b) Same only in execution-time.

c) Logical and physical addresses differ in all three binding schemes.

d) Same only in compile-time.

10. What is the main purpose of the swapping process in memory management? Main Memory

a) To remove a process from main memory.

b) To increase the size of main memory.

c) **To move a process out of main memory to a backing store and bring it back later for execution.**

d) To prevent processes from using secondary storage.

11. Which of the following statements about memory allocation and fragmentation is correct? Main Memory

a) First-fit allocates the smallest hole that is big enough. —————→ first hole that is big enough

b) Best-fit allocates the first hole that is big enough. —————→ smallest hole that is big enough

c) **Worst-fit allocates the largest available hole, producing the largest leftover hole.**

d) Compaction is used to reduce internal fragmentation only. —————→ reduce external fragmentation

12. Paging and segmentation are examples of which type of memory-management scheme?

Main Memory

a) Contiguous memory allocation schemes.

b) Non-contiguous memory allocation schemes.

c) Static memory allocation schemes.

d) Temporary storage schemes.

13. Which of the following correctly describes the difference and similarity between paging and segmentation?

Main Memory

a) Both divide memory into logical segments of variable size and use a page table.

b) Paging divides memory into fixed-size blocks, while segmentation divides it into logical segments of variable size.

c) Paging and segmentation both divide memory into fixed-size blocks called pages.

d) Paging divides memory logically, while segmentation divides it physically. —→ Opposite is right

14. Which of the following statements is TRUE about the LFU and MFU page replacement algorithms?

Virtual Memory

a) LFU replaces the page with the largest reference count, while MFU replaces the page with the smallest reference count.

b) LFU replaces the page with the smallest reference count, while MFU replaces the page with the largest reference count.

c) Both LFU and MFU replace the page that was most recently used.

d) Both LFU and MFU always behave identically to the LRU algorithm

15. Which of the following statements about thrashing and prepaging is TRUE?

Virtual Memory

a) Thrashing occurs when a process has more than enough pages in memory, leading to low page-fault rate.

b) Prepaging always improves system performance regardless of how many of the pages are actually used.

c) Thrashing occurs when a process has too few pages, causing a very high page-fault rate and constant swapping.

d) Prepaging loads all pages of a process into memory to guarantee no page faults ever occur.

Memory Management

A. Consider a system of 16 GB (2^{34} B) of memory with a frame size of 8 KB (2^{13} B) and a process P with a logical address space of size 450 KB. Assume that page N of this process is mapped to frame N+2000. Answer the following questions showing your work.

1. What is the physical address of the logical address 345678?

$$\text{Logical Address} = 345678$$

$$\text{Virtual Logical Address} = (345678 / 8192, 345678 \% 8192) = (42, 1614)$$

$$\text{Virtual Physical Address} = (2000 + 42, 1614) = (2042, 1614)$$

$$\text{Physical Address} = 2042 * 8192 + 1614 = 16729678$$

2. What is the amount of internal fragmentation, if any?

$$\text{Internal Fragmentation} = \text{Page Size} - (\text{Process Size} \% \text{Page Size})$$

$$\text{Internal Fragmentation} = 8 \text{ KB} - (450 \text{ KB} \% 8 \text{ KB}) = 8 - 2 = 6 \text{ KB}$$

3. How many pages are needed to represent this process?

$$\frac{450 \text{ KB}}{8 \text{ KB}} = 56.2 = 57$$

4. How many bits are needed to represent the following?

- Page number $\rightarrow \log_2 (\text{number of pages}) = \log_2 (57) = 6 \text{ bits}$

- Offset $\rightarrow \log_2 (\text{page size}) = \log_2 (2^{13}) = 13 \text{ bits}$

Page Table Size

$$= (\text{bits of frame} + \text{offset}) \text{ number of pages}$$

$$= (\log_2 (2^{34}) + 13) \times 57 = 1938 \text{ bits}$$

B. Consider the following segmentation table for the process P with zero internal fragmentation.

Answer the following questions showing your work.

Segment Number	Base Address	Limit
0	12100000	5400
1	14200000	7600
2	16300000	16500
3	18400000	2800

1. What is the **physical address** of the **logical address** 16500?

$$16500 > 5400 \implies 16500 - 5400 = 11100$$

$$11100 > 7600 \implies 11100 - 7600 = 3500$$

$$3500 < 16500 \implies \text{Segment (2)} \implies (2, 3500)$$

$$\text{Physical address} = 16300000 + 3500 = 16303500$$

2. What is the **logical address** of the **physical address** of 16305000?

$$16305000 - 16300000 = 5,000 \implies 5400 + 7600 + 5000 = 18000$$

3. What is the **virtual logical address** of the **physical address** 14204567?

$$14204567 - 14200000 = 4567$$

4. What is the **virtual logical address** of the **logical address** 12500?

$$12500 - 5400 = 7100 \text{ (} 7100 < 7600 \implies \text{Segment (1)} \text{)} \implies (1, 7100)$$

5. What is the **total size** of the process address space?

$$5400 + 7600 + 16500 + 2800 = 32300$$

C. Consider a **memory of size 32KB** This memory is allocated in increments of **2KB** and the memory is currently represented as follows:

Free (6KB)	Occupied (8KB)	Free (10KB)	Occupied (4KB)	Free (4KB)
---------------	-------------------	----------------	-------------------	---------------

Use the various contiguous allocation schemes (First-fit, Best-fit, and Worst-fit) to show the memory representation after satisfying the allocation requests for processes requesting sizes as given below

Process	A	B	C	D
Requested size (in KB)	3.5	2.3	5.2	1.6
	4	4	6	2

First-fit	
Best-fit Internal Fragmentation A: 4 - 3.5 = 0.5 B: 4 - 2.3 = 1.7 C: 6 - 5.2 = 0.8 D: 2 - 1.6 = 0.4	
Worst-fit	

The amount of <u>internal</u> fragmentation (in KB) for <u>Best-fit algorithm</u> is <u>0.5 + 1.7 + 0.8 + 0.4 = 3.4 KB</u>
The amount of <u>external</u> fragmentation (in KB) for <u>Worst-fit algorithm</u> is <u>2KB + 2KB = 4 KB</u>

D. Consider a system in which a process is allocated three frames in memory. Given the page requests of this process as "7, 5, 3, 1, 5, 2, 4, 6, 4, 2, 5, 7, 1, 5, 3", trace the utilization of the three frames by this process and find the number of page faults when using the Least Recently Used (**LRU**) replacement algorithm.

	7	5	3	1	5	2	4	6	4	2	5	7	1	5	3
0	7	7	7	1		1	4	4			4	7	7		3
1		5	5	5		5	5	6			5	5	5		5
2			3	3		2	2	2			2	2	1		1

Number of page Faults =**11**..... $P = \frac{11}{15} = 0.73$

Consider a system in which a process is allocated three frames in memory. Given the requests of this process as "1, 4, 5, 3, 4, 5, 6, 1, 7, 5, 6, 1, 3, 2, 6, 5, 2, 1, 5, 2", trace the utilization of the three frames by this process when using the **Optimal, FIFO, LRU** replacement algorithms. To compare the efficiency of using each algorithm, find the total number of page faults and the effective access time. Assume that the **memory access time is 100 nanoseconds** and the **average page-fault service time is 4 microseconds**.

FIFO Replacement

	1	4	5	3	4	5	6	1	7	5	6	1	3	2	6	5	2	1	5	2
0	1	1	1	3			3	3	7	7	7	1	1	1	6	6		6		2
1		4	4	4			6	6	6	5	5	5	3	3	3	5		5		5
2			5	5			5	1	1	1	6	6	6	2	2	2		1		1

Number of page faults = 16 $P = \frac{16}{20} = 0.8$

Effective Access Time (EAT) = (1 – P) x MAT + (P x APF) = $(1 - 0.8) \times 100 + (0.8 \times 4 \times 10^3)$ = 3220 ns
= 3.220 ms

Optimal Replacement

	1	4	5	3	4	5	6	1	7	5	6	1	3	2	6	5	2	1	5	2
0	1	1	1	3			3	1	7			1	3	2				2		
1		4	4	4			6	6	6			6	6	6				1		
2			5	5			5	5	5			5	5	5				5		

Number of page faults = 11 $P = \frac{11}{20} = 0.55$

Effective Access Time (EAT) = (1 – P) x MAT + (P x APF) = $(1 - 0.55) \times 100 + (0.55 \times 4 \times 10^3)$ = 2245 ns
= 2.245 ms

LRU Replacement

	1	4	5	3	4	5	6	1	7	5	6	1	3	2	6	5	2	1	5	2
0	1	1	1	3			6	6	6	5	5	5	3	3	3	5		5		
1		4	4	4			4	1	1	1	6	6	6	2	2	2		2		
2			5	5			5	5	7	7	7	1	1	1	6	6		1		

Number of page faults = 15 $P = \frac{15}{20} = 0.75$

Effective Access Time (EAT) = (1 – P) x MAT + (P x APF) = $(1 - 0.75) \times 100 + (0.75 \times 4 \times 10^3)$ = 3025 ns
= 3.025 ms

Consider a **48-bit virtual address space**. The system uses a **page size of 4KB** and **two-level hierarchical paging**. Therefore, the **page table is divided into 4KB pages** and the logical address has the following format:

P1	P2	offset
----	----	--------

Below are the relevant tables. The values are provided in hexadecimal.

Outer Page Table	
0	Page A of PT
1	Page B of PT
x	(rest are invalid)

Page A of Page Table	
0	FEE
1	3BC
2	124
3	C3D
x	(rest are invalid)

Page B of the Page Table	
0	330
1	44B
2	E55
x	(rest are invalid)

What is the size (number of bits) of each of the following logical address fields:

- Size of Offset = **12 bits**

$$\frac{2^{48}}{2^{12}} = 2^{36}$$

page size of 4KB = 2^{12} B
 Offset = $\log_2(2^{12}) = 12$

- Size of P1 = $36 - 12 =$ **24 bits**

- Size of P2 = $24 - 12 =$ **12 bits**

$$\frac{2^{36}}{2^{12}} = 2^{24}$$

What is the physical/logical address corresponding to each of the following shown logical/physical addresses. Answer "Bad logical address" if the logical address is invalid.

logical address	Physical address
000000 00374F	C3D74F
000002 0015A2	invalid
000001 002FEE	E55FEE
000001 001321	44B321
000000 001330	3BC330

Consider a computer system with a **32-bit logical address** and **4-KB page size**. The system supports up to **512 MB of physical memory**. How many entries are there in each of the following?

- A conventional, single-level page table
- An inverted page table
- The outer and inner tables for a 2-level page table where the PT is paged into 4-KB pages

Logical address = 32 bits
page size = 2^{12} bytes
Offset = $\text{Log}_2(\text{page size}) = \text{Log}_2(2^{12}) = 12$ bits
Physical memory = 512 MB = 2^{29} bytes

Conventional, single-level PT size = 2^{20} entries (since $32-12=20$ bits are used for page number)

Inverted PT size = $2^{29} / 2^{12} = 2^{17} = 128\text{K}$ entries (same as number of frames; notice that $512 \text{ MB} = 2^{29}$)

Outer table size for 2-level PT = 2^8 entries (specified by $32-12-12=8$ bits)

Inner table size for 2-level PT = $2^8 \times 2^{12}$ entries (2^8 pages each has 2^{12} entries)

Hash table entries = $2^{29} / 2^{12} = 2^{17}$ = Number of frames